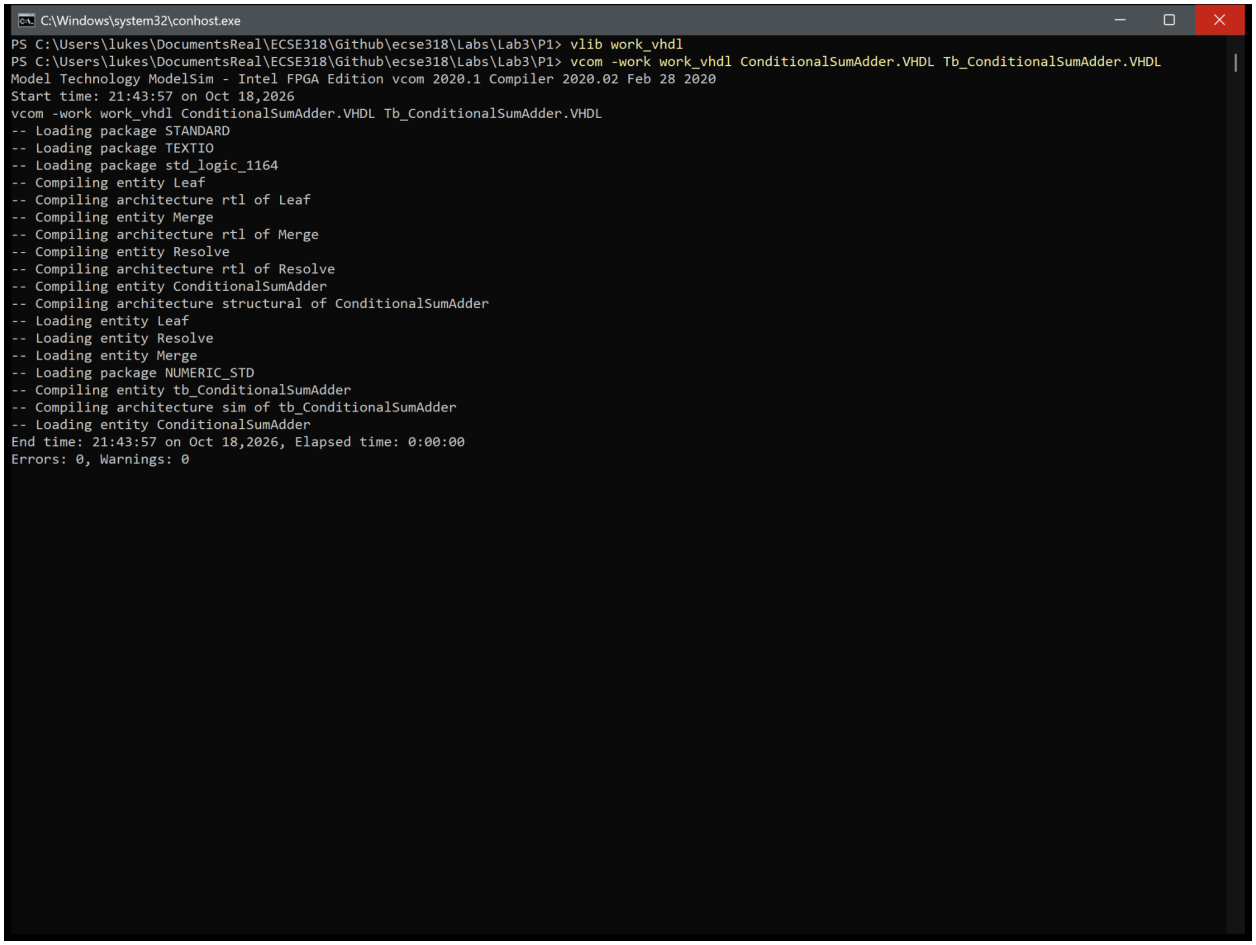
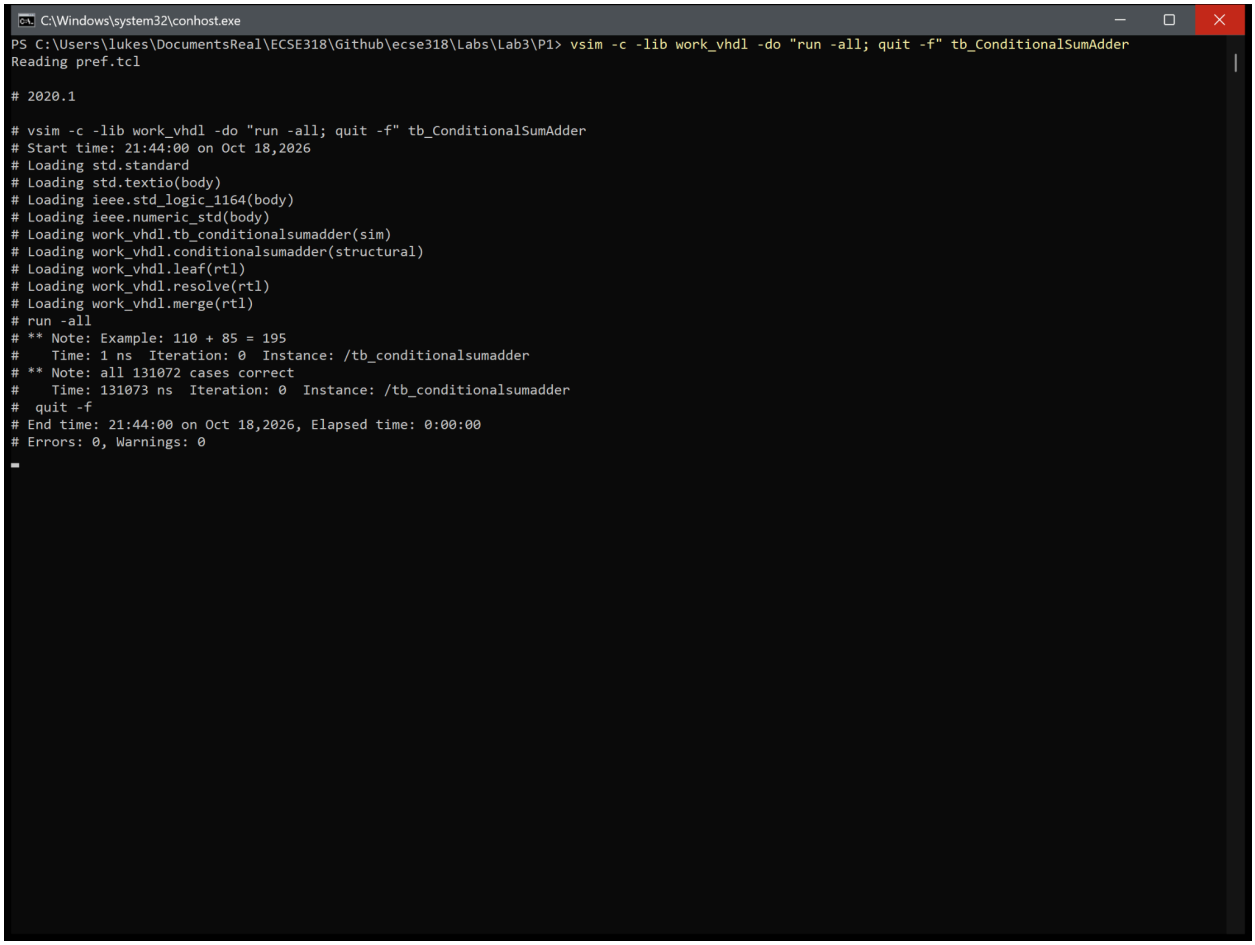
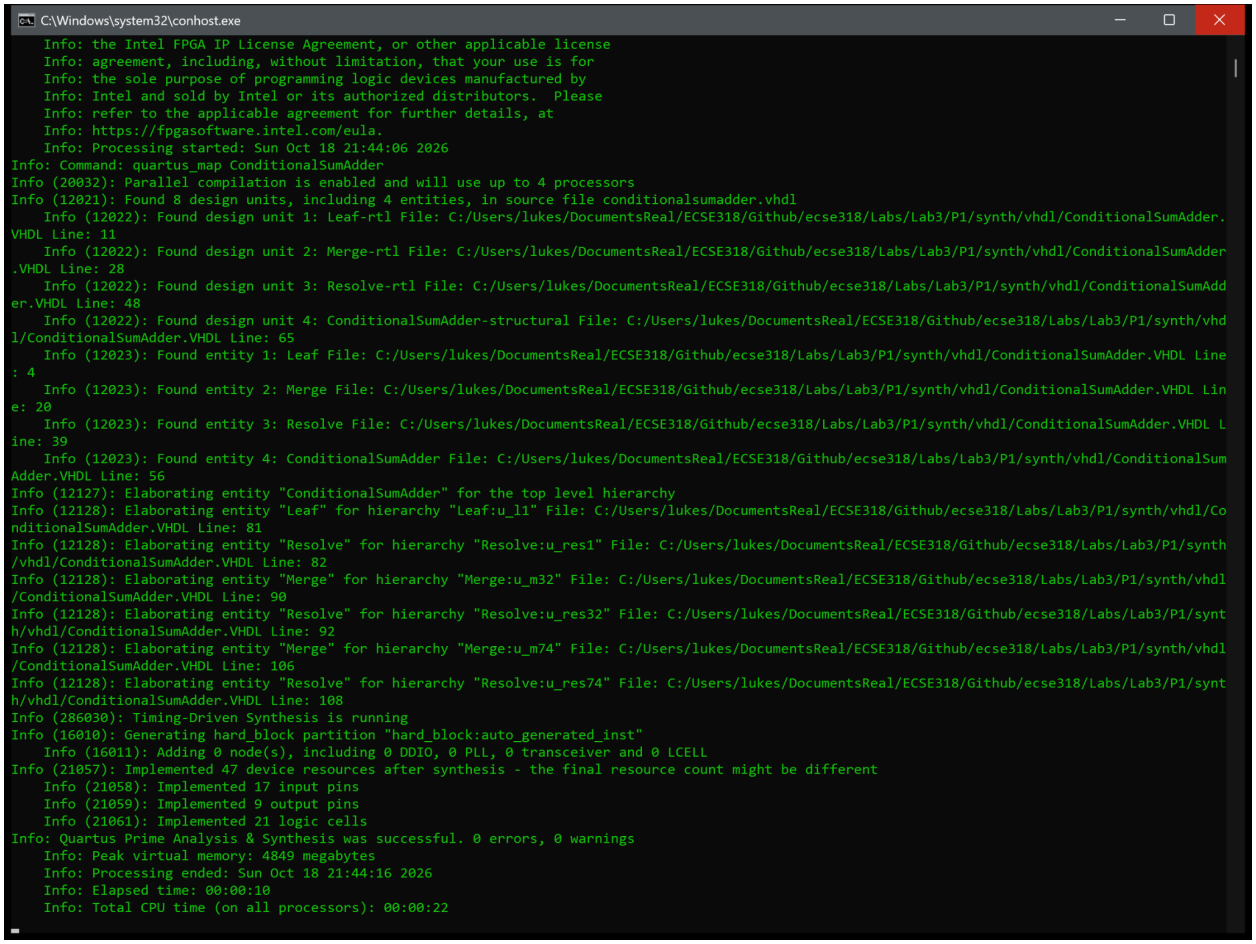
Problem 1:

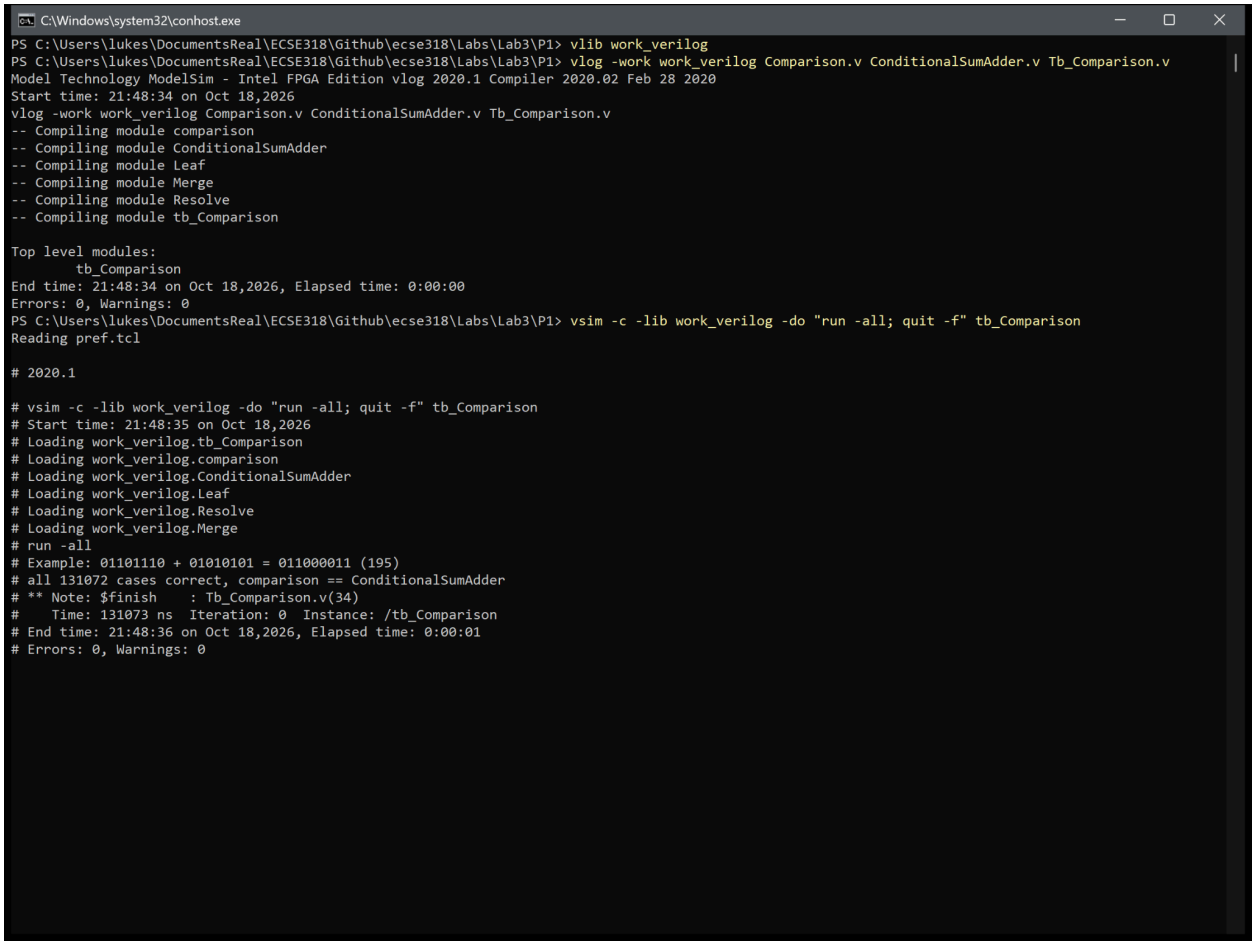
I built the adder first in verilog so that making it in VHDL was easier. The VHDL Version Passed all possible input combinations in simulation. It is also synthesized with 21 logic cells.

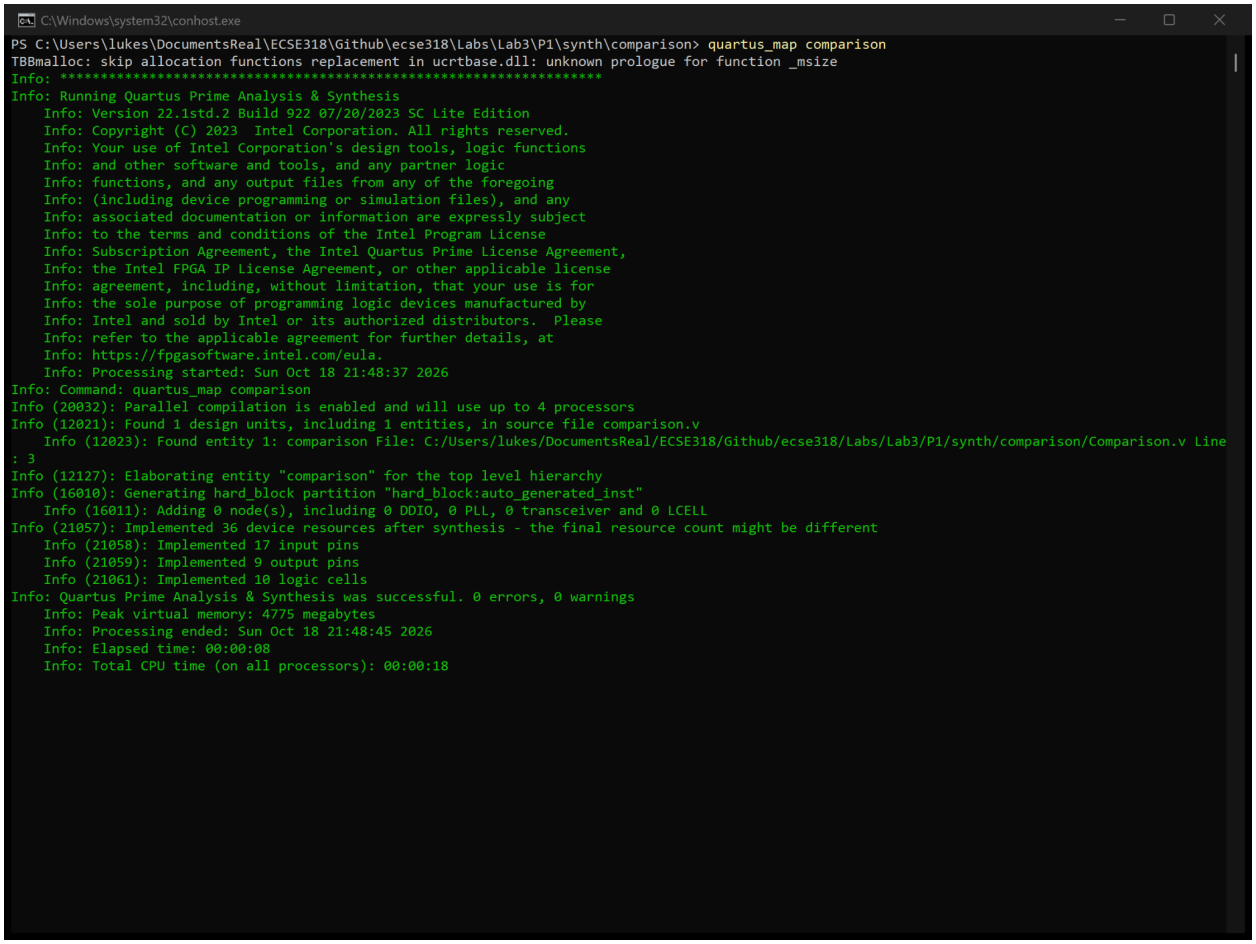



C:\Windows\system32\conhost.exe
PS C:\Users\lukes\DocumentsReal\ECSE318\Github\ecse318\Labs\Lab3\P1> vsim -c -lib work_vhdl -do "run -all; quit -f" tb_ConditionalSumAdder
Reading pref.tcl
2020.1
vsim -c -lib work_vhdl -do "run -all; quit -f" tb_ConditionalSumAdder
Start time: 21:44:00 on Oct 18,2026
Loading std.standard
Loading std.textio(body)
Loading ieee.std_logic_1164(body)
Loading ieee.numeric_std(body)
Loading work_vhdl.tb_conditionalsumadder(sim)
Loading work_vhdl.conditionalsumadder(structural)
Loading work_vhdl.leaf(rtl)
Loading work_vhdl.resolve(rtl)
Loading work_vhdl.merge(rtl)
run -all
** Note: Example: 110 + 85 = 195
Time: 1 ns Iteration: 0 Instance: /tb_conditionalsumadder
** Note: all 131072 cases correct
Time: 131073 ns Iteration: 0 Instance: /tb_conditionalsumadder
quit -f
End time: 21:44:00 on Oct 18,2026, Elapsed time: 0:00:00
Errors: 0, Warnings: 0



For comparison I also just made a ripple carry adder in verilog and it was about half the size but a lot faster.


C:\Windows\system32\conhost.exe
PS C:\Users\lukes\DocumentsReal\ECSE318\Github\ecse318\Labs\Lab3\P1> vlib work_verilog
PS C:\Users\lukes\DocumentsReal\ECSE318\Github\ecse318\Labs\Lab3\P1> vlog -work work_verilog Comparison.v ConditionalSumAdder.v Tb_Comparison.v
Model Technology ModelSim - Intel FPGA Edition vlog 2020.1 Compiler 2020.02 Feb 28 2020
Start time: 21:48:34 on Oct 18,2026
vlog -work work_verilog Comparison.v ConditionalSumAdder.v Tb_Comparison.v
-- Compiling module comparison
-- Compiling module ConditionalSumAdder
-- Compiling module Leaf
-- Compiling module Merge
-- Compiling module Resolve
-- Compiling module tb_Comparison
Top level modules:
tb_Comparison
End time: 21:48:34 on Oct 18,2026, Elapsed time: 0:00:00
Errors: 0, Warnings: 0
PS C:\Users\lukes\DocumentsReal\ECSE318\Github\ecse318\Labs\Lab3\P1> vsim -c -lib work_verilog -do "run -all; quit -f" tb_Comparison
Reading pref.tcl
2020.1
vsim -c -lib work_verilog -do "run -all; quit -f" tb_Comparison
Start time: 21:48:35 on Oct 18,2026
Loading work_verilog.tb_Comparison
Loading work_verilog.comparison
Loading work_verilog.ConditionalSumAdder
Loading work_verilog.Leaf
Loading work_verilog.Resolve
Loading work_verilog.Merge
run -all
Example: 01101110 + 01010101 = 011000011 (195)
all 131072 cases correct, comparison == ConditionalSumAdder
** Note: $finish : Tb_Comparison.v(34)
Time: 131073 ns Iteration: 0 Instance: /tb_Comparison
End time: 21:48:36 on Oct 18,2026, Elapsed time: 0:00:01
Errors: 0, Warnings: 0


C:\Windows\system32\conhost.exe
PS C:\Users\lukes\DocumentsReal\ECSE318\Github\ecse318\Labs\Lab3\P1\synth\comparison> quartus_map comparison
TBBmalloc: skip allocation functions replacement in ucrtbase.dll: unknown prologue for function _msize
Info: ***
Info: Running Quartus Prime Analysis & Synthesis
Info: Version 22.1std.2 Build 922 07/20/2023 SC Lite Edition
Info: Copyright (C) 2023 Intel Corporation. All rights reserved.
Info: Your use of Intel Corporation's design tools, logic functions
Info: and other software and tools, and any partner logic
Info: functions, and any output files from any of the foregoing
Info: (including device programming or simulation files), and any
Info: associated documentation or information are expressly subject
Info: to the terms and conditions of the Intel Program License
Info: Subscription Agreement, the Intel Quartus Prime License Agreement,
Info: the Intel FPGA IP License Agreement, or other applicable license
Info: agreement, including, without limitation, that your use is for
Info: the sole purpose of programming logic devices manufactured by
Info: Intel and sold by Intel or its authorized distributors. Please
Info: refer to the applicable agreement for further details, at
Info: https://fpgasoftware.intel.com/eula.
Info: Processing started: Sun Oct 18 21:48:37 2026
Info: Command: quartus_map comparison
Info (20032): Parallel compilation is enabled and will use up to 4 processors
Info (12021): Found 1 design units, including 1 entities, in source file comparison.v
Info (12023): Found entity 1: comparison File: C:/Users/lukes/DocumentsReal/ECSE318/Github/ecse318/Labs/Lab3/P1/synth/comparison/Comparison.v Line
: 3
Info (12127): Elaborating entity "comparison" for the top level hierarchy
Info (16010): Generating hard_block partition "hard_block:auto_generated_inst"
Info (16011): Adding 0 node(s), including 0 DDIO, 0 PLL, 0 transceiver and 0 LCELL
Info (21057): Implemented 36 device resources after synthesis - the final resource count might be different
Info (21058): Implemented 17 input pins
Info (21059): Implemented 9 output pins
Info (21061): Implemented 10 logic cells
Info: Quartus Prime Analysis & Synthesis was successful. 0 errors, 0 warnings
Info: Peak virtual memory: 4775 megabytes
Info: Processing ended: Sun Oct 18 21:48:45 2026
Info: Elapsed time: 00:00:08
Info: Total CPU time (on all processors): 00:00:18

Problem 4:

Each of the 52 cards were represented with a 6-bit vector where the two most significant bits encoded the suit of the card and the four least significant bits represented the value of the card The 6-bit encoding minimizes resource usage while clearly distinguishing all 52 cards and separating suit and value simplifies move validation logic.

The tableau columns were represented as a 7 x 8 2d array with each index containing a 6 bit card. An array register was used to tracks the number of cards in each column (allowing columns to grow or shrink as cards are moved).

The free cells were represented as an array for the four free cells with each index containing a 6 bit card. An bitmask was used to track which free cells were occupied.

The home cells were represented as an a 13 x 4 2d array with each index containing a 6 bit card. An array register was used to track the number of cards in each home cell.

The win signal is updated and transitions to 1 when all 52 cards are in their home cells.

The design enforces the FreeCell rules and accounts for edge cases such as: Moving a card to an empty tableau column is always valid, Only Aces can be moved to empty home cells. Attempting to move a card to a full free cell is rejected, Moves that violate FreeCell rules are ignored, and the state remains unchanged.

The design uses the following registers, which correspond to 717 Total latches in a synthesized design:

Tableau -> 8 columns × 7 cards × 6 bits = 336 latches

Column Count -> 8 columns × 3 bits = 24 latches

Free cells -> 4 free cells × 6 bits = 24 latches

Free cell Card Bitmask -> 4 bits = 4 latches

Home cells -> 4 suits × 13 cards × 6 bits = 312 latches

Home Count -> 4 suits × 4 bits = 16 latches

Win Signal -> 1 bit = 1 latches